

Core Python Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Core Python Programming course is designed to build a strong foundation in Python programming, problem-solving, object-oriented programming, functions, collections, modules, exception handling, file handling, iterators, generators, decorators, comprehensions, regular expressions, testing, and practical application development.

The course begins with programming fundamentals and Python syntax, then progresses through control flow, functions, data structures, modules, object-oriented programming, file operations, error handling, functional programming concepts, iterators, generators, decorators, type hints, virtual environments, testing, debugging, and project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and developers who want to build a strong Python foundation before moving to web development, automation, data science, machine learning, AI, scripting, DevOps, testing, or backend development.

Course Objectives

By the end of this course, learners will be able to:

- Understand Python programming fundamentals.
- Write, execute, debug, and maintain Python programs.
- Work with variables, data types, operators, conditions, loops, strings, and collections.
- Design reusable functions and modules.
- Apply object-oriented programming principles.
- Handle exceptions and build robust applications.
- Work with files and directories.
- Use comprehensions, iterators, generators, decorators, and context managers.
- Understand modules, packages, and virtual environments.
- Use regular expressions and standard library utilities.
- Apply type hints and modern Python coding practices.
- Write unit tests for Python applications.
- Use Git and GitHub for version control.
- Build console-based mini projects and capstone projects.
- Prepare for Python programming interviews and coding assessments.

Module 1: Introduction to Programming and Python

Topics

- Introduction to Computer Programming
- What is a Programming Language?
- Compiler vs Interpreter
- High-Level vs Low-Level Languages
- Introduction to Python
- History and Evolution of Python
- Features of Python
- Applications of Python
- Why Learn Python?
- Python 2 vs Python 3
- Python Implementations Overview
- CPython
- PyPy
- Jython

- IronPython
- Python Versions Overview
- Python Enhancement Proposals
- PEP 8 Introduction
- Python Documentation and Ecosystem

Module 2: Python Installation and Development Environment

Topics

- Installing Python
- Checking Python Version
- Python Interpreter
- Python REPL
- Running Python Scripts
- .py Files
- IDLE
- Visual Studio Code
- PyCharm
- Jupyter Notebook Overview
- Terminal / Command Prompt
- python and python3 Commands
- pip Introduction
- Python Package Index
- Virtual Environments Overview
- Project Folder Structure

Practical Exercises

- Hello World Program
- Run Python in REPL
- Run Python Script from Terminal
- Create First Python Project in IDE

Module 3: Python Syntax and Program Structure

Topics

- Python Syntax
- Indentation
- Statements
- Expressions
- Comments
- Single-Line Comments
- Multi-Line Documentation Strings
- Keywords
- Identifiers
- Naming Conventions
- Case Sensitivity
- Code Blocks
- Line Continuation
- Semicolons
- PEP 8 Coding Style
- Pythonic Code Overview

Module 4: Variables and Data Types

Topics

- Variables
- Dynamic Typing
- Variable Assignment
- Multiple Assignment
- Multiple Variables
- Constants Convention
- Built-In Data Types
- int
- float
- complex
- bool
- str
- list
- tuple
- set
- dict
- range
- bytes
- bytearray
- None
- Mutable vs Immutable Types
- Type Checking
- type()
- isinstance()
- Object Identity
- id()
- Type Conversion
- Explicit Casting
- Numeric Conversion
- String Conversion

Practical Exercises

- Display Data Types
- Convert Between Numeric Types
- Type Checking Programs
- Swap Variables
- Multiple Assignment Examples

Module 5: Operators and Expressions**Topics**

- Arithmetic Operators
- Comparison Operators
- Logical Operators
- Assignment Operators
- Compound Assignment Operators
- Bitwise Operators
- Membership Operators
- in
- not in
- Identity Operators
- is

- is not
- Operator Precedence
- Operator Associativity
- Expressions
- Truthy and Falsy Values
- Chained Comparisons
- Conditional Expressions

Practical Exercises

- Even or Odd Number
- Largest of Two Numbers
- Leap Year
- Bitwise Operations
- Membership Testing

Module 6: Input and Output

Topics

- `print()`
- `input()`
- Reading User Input
- Converting User Input
- Multiple Inputs
- Formatted Output
- String Concatenation
- % Formatting Overview
- `str.format()`
- f-Strings
- Format Specifiers
- Number Formatting
- Alignment and Width
- Input Validation Basics

Practical Exercises

- Student Details Input
- Arithmetic Calculator
- Formatted Invoice
- Mark Sheet
- Currency Formatting

Module 7: Decision-Making Statements

Topics

- `if`
- `if-else`
- `if-elif-else`
- Nested `if`
- Multiple Conditions
- Logical Expressions
- Conditional Expressions
- `match-case` Overview
- Pattern Matching Basics
- Best Practices for Decision Logic

Practical Exercises

- Grade Calculator
- Positive, Negative, or Zero
- Vowel or Consonant
- Menu-Driven Calculator
- Electricity Bill Calculator

Module 8: Looping Statements**Topics**

- while
- for
- Iterating Over Sequences
- range()
- Nested Loops
- Infinite Loops
- break
- continue
- pass
- Loop else
- enumerate()
- zip()
- Iterating Dictionaries
- Best Practices for Loops

Practical Exercises

- Multiplication Table
- Sum of Natural Numbers
- Factorial
- Fibonacci Series
- Reverse a Number
- Palindrome Number
- Armstrong Number
- Prime Number
- Number Patterns
- Star Patterns

Module 9: Strings**Topics**

- String Creation
- String Immutability
- Single, Double, and Triple Quotes
- Escape Sequences
- Raw Strings
- String Indexing
- Positive Indexing
- Negative Indexing
- String Slicing
- String Concatenation
- String Repetition
- Membership Testing
- Important String Methods

- `lower()`
- `upper()`
- `title()`
- `capitalize()`
- `strip()`
- `lstrip()`
- `rstrip()`
- `replace()`
- `split()`
- `join()`
- `find()`
- `index()`
- `startswith()`
- `endswith()`
- `count()`
- `isalpha()`
- `isdigit()`
- `isalnum()`
- String Formatting
- f-Strings
- String Comparison
- Unicode Basics

Practical Exercises

- Reverse a String
- Check Palindrome
- Count Vowels and Consonants
- Character Frequency
- Word Count
- Remove Spaces
- Find Substring

Module 10: Lists

Topics

- Introduction to Lists
- List Creation
- List Indexing
- List Slicing
- Mutable Nature of Lists
- Adding Elements
- `append()`
- `extend()`
- `insert()`
- Removing Elements
- `remove()`
- `pop()`
- `clear()`
- Updating Elements
- Searching Lists
- Sorting Lists
- `sort()`
- `sorted()`

- Reversing Lists
- Copying Lists
- Shallow Copy Basics
- Nested Lists
- List Traversal
- List Comprehensions Introduction

Practical Exercises

- Sum and Average
- Maximum and Minimum
- Reverse a List
- Remove Duplicates
- Sort Student Marks
- Matrix Representation Using Lists

Module 11: Tuples

Topics

- Introduction to Tuples
- Tuple Creation
- Single-Element Tuples
- Tuple Packing
- Tuple Unpacking
- Nested Tuples
- Tuple Indexing
- Tuple Slicing
- Immutability
- Tuple Methods
- Returning Multiple Values
- Swapping Variables Using Tuples
- Lists vs Tuples

Practical Exercises

- Store Coordinates
- Return Multiple Values
- Tuple Unpacking
- Nested Tuple Processing

Module 12: Sets and Frozensets

Topics

- Introduction to Sets
- Creating Sets
- Unique Elements
- Adding Elements
- Removing Elements
- Set Membership
- Set Operations
- Union
- Intersection
- Difference
- Symmetric Difference
- Subset

- Superset
- Disjoint Sets
- Set Comprehension
- frozenset
- Lists vs Tuples vs Sets

Practical Exercises

- Remove Duplicate Values
- Common Elements Between Lists
- Set Difference
- Unique Student Names

Module 13: Dictionaries

Topics

- Introduction to Dictionaries
- Key-Value Pairs
- Creating Dictionaries
- Accessing Values
- Updating Values
- Adding Entries
- Removing Entries
- Dictionary Methods
- keys()
- values()
- items()
- get()
- pop()
- popitem()
- update()
- setdefault()
- Iterating Dictionaries
- Nested Dictionaries
- Dictionary Comprehensions
- Dictionary Keys and Hashability
- Ordered Nature of Modern Dictionaries

Practical Exercises

- Student Marks Dictionary
- Word Frequency Counter
- Employee Directory
- Inventory Data
- Nested Student Records

Module 14: Functions

Topics

- Introduction to Functions
- Defining Functions
- Calling Functions
- Parameters
- Arguments
- Return Values

- Multiple Return Values
- Positional Arguments
- Keyword Arguments
- Default Arguments
- Variable-Length Arguments
- `*args`
- `**kwargs`
- Keyword-Only Arguments
- Positional-Only Arguments Overview
- Local Variables
- Global Variables
- `global`
- `nonlocal`
- Scope
- LEGB Rule
- Recursive Functions
- Function Documentation
- Docstrings
- Type Hints Introduction

Practical Exercises

- Function-Based Calculator
- Factorial Using Recursion
- Fibonacci Using Recursion
- GCD and LCM
- Flexible Function Using `*args`

Module 15: Lambda Functions and Functional Programming Basics

Topics

- Lambda Expressions
- Lambda Syntax
- Lambda with One Argument
- Lambda with Multiple Arguments
- Lambda with Sorting
- `map()`
- `filter()`
- `reduce()`
- `functools.reduce`
- `any()`
- `all()`
- `zip()`
- `enumerate()`
- Higher-Order Functions
- First-Class Functions

Practical Exercises

- Square Numbers Using Lambda
- Filter Even Numbers
- Sort Records by Custom Key
- Calculate Total Using Reduce

Module 16: Comprehensions

Topics

- List Comprehensions
- Conditional List Comprehensions
- Nested List Comprehensions
- Set Comprehensions
- Dictionary Comprehensions
- Generator Expressions
- Comprehensions vs Traditional Loops
- Readability Considerations

Practical Exercises

- Generate Squares
- Filter Even Numbers
- Flatten a Matrix
- Build Dictionary from Two Lists

Module 17: Modules**Topics**

- Introduction to Modules
- Creating Modules
- Importing Modules
- `import`
- `from ... import`
- Aliases
- `dir()`
- `__name__`
- `if __name__ == "__main__"`
- Module Search Path
- Built-In Modules
- Custom Modules
- Reusing Code Across Files

Practical Exercises

- Create Math Utility Module
- Create String Utility Module
- Import and Reuse Custom Modules

Module 18: Packages**Topics**

- Introduction to Packages
- Package Folder Structure
- `__init__.py`
- Creating Packages
- Subpackages
- Absolute Imports
- Relative Imports
- Package Organization
- Reusable Python Components
- Namespace Packages Overview

Practical Exercises

- Create Utility Package
- Create Models and Services Packages
- Organize Multi-File Application

Module 19: Python Standard Library Essentials

Topics

- math
- random
- statistics
- datetime
- time
- calendar
- os
- sys
- pathlib
- shutil
- collections
- itertools
- functools
- json
- csv
- re
- logging
- uuid
- decimal
- fractions

Practical Exercises

- Random Number Generator
- Date Calculator
- File System Utility
- Statistics Calculator

Module 20: Exception Handling

Topics

- Errors vs Exceptions
- Syntax Errors
- Runtime Exceptions
- Exception Hierarchy
- try
- except
- Multiple except Blocks
- Catching Multiple Exceptions
- else
- finally
- raise
- Custom Exceptions
- Exception Chaining Overview
- Assertions
- Best Practices for Exception Handling

Practical Exercises

- Divide-by-Zero Handling
- Invalid Age Exception
- File Handling Exception
- Custom Banking Exception

Module 21: File Handling

Topics

- Files and Streams
- Opening Files
- `open()`
- File Modes
- Read
- Write
- Append
- Binary
- Reading Files
- `read()`
- `readline()`
- `readlines()`
- Writing Files
- Appending Files
- Closing Files
- `with` Statement
- Context Managers
- File Pointer
- `seek()`
- `tell()`
- Text vs Binary Files
- File Encoding
- Working with Paths
- File and Directory Operations

Practical Exercises

- Write Data to File
- Read Data from File
- Copy File
- Count Words and Lines
- File-Based Student Record System

Module 22: CSV and JSON Handling

CSV

- csv Module
- CSV Reader
- CSV Writer
- DictReader
- DictWriter

JSON

- JSON Format
- `json.dumps()`
- `json.loads()`

- `json.dump()`
- `json.load()`
- Python Objects to JSON
- JSON to Python Objects
- Pretty Printing

Practical Exercises

- Read Student CSV
- Write Employee CSV
- Store Application Data in JSON
- Load JSON Configuration

Module 23: Object-Oriented Programming Fundamentals

Topics

- Procedural Programming vs Object-Oriented Programming
- OOP Concepts
- Class
- Object
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Defining Classes
- Creating Objects
- Instance Attributes
- Class Attributes
- Instance Methods
- `self`
- Object State and Behavior
- Attribute Access
- Naming Conventions

Practical Exercises

- Student Class
- Employee Class
- Product Class
- Bank Account Class

Module 24: Constructors and Object Initialization

Topics

- `__init__()`
- Constructor-Like Initialization in Python
- Object Creation
- Instance Attribute Initialization
- Default Values
- Optional Constructor Arguments
- Class-Level Defaults
- Object Lifecycle Basics
- `__new__()` Overview
- Destruction and `__del__()` Overview

Practical Exercises

- Student Initialization
- Employee Object Creation
- Product Constructor Validation

Module 25: Encapsulation and Properties

Topics

- Encapsulation
- Public Attributes
- Protected Naming Convention
- Private Name Mangling
- Getter and Setter Methods
- `property()`
- `@property`
- Setter Decorator
- Deleter Decorator
- Read-Only Properties
- Validation Through Properties
- Pythonic Encapsulation

Practical Exercises

- Bank Account with Property Validation
- Employee Salary Property
- Product Price Validation

Module 26: Inheritance

Topics

- Base Class
- Derived Class
- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- `super()`
- Method Overriding
- Constructor Chaining
- Method Resolution Order
- MRO
- Diamond Problem
- Composition vs Inheritance
- IS-A vs HAS-A Relationships

Practical Exercises

- Person and Student
- Employee Hierarchy
- Vehicle Hierarchy
- Multiple Inheritance Example

Module 27: Polymorphism

Topics

- Polymorphism in Python

- Method Overriding
- Duck Typing
- Operator Overloading
- Polymorphic Functions
- Common Interfaces
- Abstract Base Classes
- Runtime Polymorphism
- Protocol-Oriented Thinking Overview

Practical Exercises

- Shape Area Calculation
- Payment Processing
- Common speak () Interface
- Operator Overloading Example

Module 28: Abstraction and Abstract Base Classes

Topics

- Abstraction
- abc Module
- ABC
- @abstractmethod
- Abstract Classes
- Abstract Methods
- Concrete Methods
- Interface-Like Design in Python
- Programming to Abstractions

Practical Exercises

- Abstract Shape
- Payment Interface
- Notification Service Abstraction

Module 29: Special Methods and Operator Overloading

Topics

- Magic / Dunder Methods
- `__str__`
- `__repr__`
- `__len__`
- `__eq__`
- `__lt__`
- `__add__`
- `__sub__`
- `__mul__`
- `__contains__`
- `__getitem__`
- `__setitem__`
- `__iter__`
- `__call__`
- Operator Overloading
- Custom Object Behavior

Practical Exercises

- Complex Number Class
- Custom Collection Class
- Student Comparison
- Vector Addition

Module 30: Class Methods and Static Methods

Topics

- Instance Methods
- `@classmethod`
- `cls`
- Alternative Constructors
- `@staticmethod`
- Utility Methods
- Instance vs Class vs Static Methods
- Choosing the Correct Method Type

Practical Exercises

- Alternative Date Constructor
- Employee Factory Method
- Static Validation Utility

Module 31: Dataclasses

Topics

- Introduction to dataclasses
- `@dataclass`
- Auto-Generated `__init__`
- Auto-Generated `__repr__`
- Equality
- Default Values
- `field()`
- Frozen Dataclasses
- Ordering
- Dataclasses vs Traditional Classes

Practical Exercises

- Student Dataclass
- Product Dataclass
- Immutable Configuration Model

Module 32: Iterators

Topics

- Iterable vs Iterator
- `iter()`
- `next()`
- Iterator Protocol
- `__iter__`
- `__next__`
- StopIteration
- Custom Iterators
- Lazy Evaluation

Practical Exercises

- Custom Range Iterator
- Reverse Iterator
- Even Number Iterator

Module 33: Generators**Topics**

- Generator Functions
- yield
- Generator Objects
- Lazy Evaluation
- Generator Expressions
- Memory Efficiency
- yield from
- Generator Pipelines
- Generators vs Lists

Practical Exercises

- Fibonacci Generator
- File Line Generator
- Infinite Sequence Generator

Module 34: Decorators**Topics**

- Functions as First-Class Objects
- Nested Functions
- Closures
- Function Decorators
- @decorator Syntax
- Decorators with Arguments
- Multiple Decorators
- Preserving Metadata
- `functools.wraps`
- Class Decorators Overview
- Practical Use Cases
- Logging
- Timing
- Authentication
- Validation

Practical Exercises

- Logging Decorator
- Execution Time Decorator
- Authorization Simulation
- Retry Decorator

Module 35: Context Managers**Topics**

- Resource Management
- with Statement

- Built-In Context Managers
- Custom Context Managers
- `__enter__`
- `__exit__`
- `contextlib`
- `@contextmanager`
- Safe Resource Cleanup

Practical Exercises

- Custom File Context Manager
- Timer Context Manager
- Resource Cleanup Example

Module 36: Regular Expressions

Topics

- Introduction to Regular Expressions
- `re` Module
- `match()`
- `search()`
- `findall()`
- `finditer()`
- `split()`
- `sub()`
- Character Classes
- Quantifiers
- Groups
- Anchors
- Raw Strings for Regex
- Compiled Patterns
- Validation Patterns

Practical Exercises

- Email Validation
- Mobile Number Validation
- Password Validation
- Extract Numbers
- Search and Replace

Module 37: Date and Time Programming

Topics

- `datetime` Module
- `date`
- `time`
- `datetime`
- `timedelta`
- Current Date and Time
- Parsing Dates
- Formatting Dates
- `strftime`
- `strptime`
- Date Arithmetic

- Comparing Dates
- Time Zones Overview
- zoneinfo Overview

Practical Exercises

- Age Calculator
- Date Difference
- Appointment Scheduler
- Format Date and Time

Module 38: Collections Module

Topics

- collections
- Counter
- defaultdict
- deque
- namedtuple
- ChainMap
- Ordered Dictionary Historical Context
- Use Cases

Practical Exercises

- Word Frequency Using Counter
- Queue Using Deque
- Default Dictionary Grouping

Module 39: itertools and functools

itertools

- count
- cycle
- repeat
- chain
- product
- permutations
- combinations
- groupby

functools

- reduce
- partial
- lru_cache
- wraps
- singledispatch Overview

Practical Exercises

- Generate Combinations
- Function Partial Application
- Memoization Using lru_cache

Module 40: Type Hints and Static Typing Basics

Topics

- Dynamic Typing vs Type Hints
- Variable Annotations
- Function Parameter Types
- Return Types
- Built-In Generic Types
- `list[str]`
- `dict[str, int]`
- Optional Values
- Union Types
- Any
- Callable
- Type Aliases
- TypedDict Overview
- Protocols Overview
- Static Type Checkers
- mypy Overview
- Benefits and Limitations

Practical Exercises

- Add Type Hints to Functions
- Type-Annotated Data Models
- Run Static Type Checking

Module 41: Virtual Environments and Package Management

Topics

- Why Virtual Environments?
- `venv`
- Creating Virtual Environment
- Activating Virtual Environment
- Deactivating Environment
- `pip`
- Installing Packages
- Uninstalling Packages
- Listing Packages
- `pip freeze`
- `requirements.txt`
- Dependency Management Basics
- PyPI
- Project Isolation

Practical Exercises

- Create Virtual Environment
- Install External Package
- Generate `requirements.txt`

Module 42: Command-Line Arguments

Topics

- `sys.argv`
- Command-Line Parameters
- Basic CLI Programs
- `argparse` Introduction

- Positional Arguments
- Optional Arguments
- Flags
- Help Messages
- CLI Best Practices

Practical Exercises

- Command-Line Calculator
- File Search Utility
- Argument-Based Greeting Program

Module 43: Logging

Topics

- Why Logging?
- Logging Module
- Logging Levels
- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL
- Basic Configuration
- Log Formatting
- Logging to File
- Multiple Handlers Overview
- Logging Exceptions
- Logging Best Practices

Practical Exercises

- Application Logger
- Error Logging
- File-Based Logging

Module 44: Debugging and Error Diagnosis

Topics

- Syntax Errors
- Runtime Errors
- Logical Errors
- Reading Tracebacks
- Print Debugging
- IDE Debuggers
- Breakpoints
- Step Into
- Step Over
- Variable Inspection
- pdb
- Assertions
- Logging for Debugging
- Common Python Errors
- TypeError
- ValueError

- KeyError
- IndexError
- AttributeError
- NameError
- ImportError
- ModuleNotFoundError

Module 45: Unit Testing

Topics

- Importance of Testing
- Test Cases
- Unit Testing Concepts
- unittest
- Test Classes
- Test Methods
- Assertions
- Setup and Teardown
- Testing Exceptions
- pytest Overview
- Fixtures Overview
- Parameterized Testing Overview
- Mocking Introduction
- Test-Driven Development Introduction

Practical Exercises

- Test Calculator Functions
- Test String Utilities
- Test Bank Account Class
- Test Exceptions

Module 46: Python Memory Management

Topics

- Python Object Model
- References
- Reference Counting
- Garbage Collection
- Cyclic Garbage
- Mutable vs Immutable Objects
- Shallow Copy
- Deep Copy
- copy Module
- Object Identity
- Interning Overview
- Memory Efficiency Basics

Practical Exercises

- Shallow vs Deep Copy
- Object Identity Demonstrations
- Mutable Object Behavior

Module 47: Concurrency Overview

Topics

- Concurrency vs Parallelism
- Process vs Thread
- Global Interpreter Lock Overview
- `threading`
- Creating Threads
- Joining Threads
- Shared State
- Race Conditions
- Locking Basics
- `multiprocessing` Overview
- Process-Based Parallelism
- When to Use Threads vs Processes

Practical Exercises

- Create Multiple Threads
- Threaded Task Simulation
- Process-Based Calculation

Module 48: Asynchronous Programming Introduction**Topics**

- Synchronous vs Asynchronous Programming
- `async`
- `await`
- Coroutines
- Event Loop
- `asyncio`
- Creating Tasks
- Awaiting Tasks
- `asyncio.gather`
- Async Sleep
- I/O-Bound Workloads
- Async Programming Use Cases

Practical Exercises

- Simple Coroutine
- Run Multiple Async Tasks
- Simulated Async API Calls

Module 49: Database Programming Basics with SQLite**Topics**

- Introduction to Databases
- Relational Database Basics
- SQL Overview
- SQLite
- `sqlite3` Module
- Database Connection
- Cursor
- Creating Tables
- Insert
- Select

- Update
- Delete
- Parameterized Queries
- SQL Injection Awareness
- Transactions
- Commit and Rollback
- Closing Connections

Practical Exercises

- Student Database
- Employee CRUD Application
- Inventory Database

Module 50: Clean Code and Professional Python Practices

Topics

- PEP 8
- Naming Conventions
- Readable Code
- Small Functions
- Separation of Concerns
- DRY Principle
- KISS Principle
- Avoiding Global State
- Exception Handling Best Practices
- Resource Management
- Pythonic Idioms
- List Comprehensions vs Loops
- Type Hints
- Docstrings
- Documentation
- Code Reviews
- Static Analysis Overview
- Linters
- Ruff Overview
- Flake8 Overview
- Formatting
- Black Overview
- Import Sorting Overview

Module 51: Problem-Solving and Algorithmic Thinking

Topics

- Understanding Problem Statements
- Identifying Inputs and Outputs
- Constraints
- Pseudocode
- Dry Runs
- Edge Cases
- Time Complexity Introduction
- Space Complexity Introduction
- Big-O Notation Basics
- Recursion

- Searching
- Sorting
- Choosing Appropriate Data Structures
- Common Problem-Solving Patterns

Practice Categories

- Number Problems
- Pattern Problems
- String Problems
- List Problems
- Dictionary Problems
- Set Problems
- Recursion Problems
- OOP Problems
- File Handling Problems

Module 52: Data Structures Using Python

Topics

- Lists as Dynamic Arrays
- Tuples
- Sets
- Dictionaries
- Stacks
- Queues
- deque
- Linked List Concepts
- Trees Overview
- Binary Search Tree Introduction
- Heaps Overview
- heapq
- Graphs Overview
- Hashing Concepts

Algorithms

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort Overview
- Quick Sort Overview

Module 53: Git and GitHub for Python Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`

- `git log`
- Branching Basics
- Merging
- GitHub Repository Creation
- Push and Pull
- Pull Requests Overview
- README Documentation
- `.gitignore`
- Python-Specific `.gitignore`
- Virtual Environment Exclusion
- Building a Programming Portfolio

Module 54: AI-Assisted Python Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining Python Code
- Understanding Tracebacks
- Debugging with AI Assistance
- Generating Unit Tests
- Refactoring Suggestions
- Code Review with AI
- Improving Type Hints
- Reviewing Exception Handling
- Generating Documentation
- Security Risks of Blindly Using Generated Code
- Validating AI-Generated Solutions
- Prompting Techniques for Python Programming

Recommended Practice

Students should learn to:

- Ask AI for explanations rather than only final solutions.
- Verify generated code manually.
- Run tests and validate edge cases.
- Review dependency choices.
- Check performance of generated code.
- Verify file and database operations.
- Understand every generated line before submission.

Module 55: Mini Projects

Students should complete multiple mini projects such as:

- Calculator Application
- Student Grade Management System
- Employee Management System
- Library Management System
- Banking Management System
- Inventory Management System
- Contact Management System
- Quiz Application
- Expense Tracker
- Task Management Application

- Number Guessing Game
- Password Generator
- File Organizer
- CSV Data Analyzer
- JSON-Based Address Book
- SQLite Student Management System
- Log File Analyzer
- Command-Line To-Do Application

Module 56: Capstone Project

Learners should build one complete Python application using the concepts covered in the course.

Suggested Capstone Projects

- Library Management System
- Hospital Management System
- Student Information System
- Banking Management System
- Employee Payroll System
- Inventory and Billing System
- Hotel Reservation System
- Task Management System
- Personal Expense Manager
- SQLite-Based CRUD Application

Capstone Requirements

The project should demonstrate:

- Modular programming
- Functions
- Object-oriented design
- Classes and objects
- Encapsulation
- Inheritance where appropriate
- Exception handling
- Collections
- File handling or database persistence
- JSON or CSV processing
- Logging
- Input validation
- Type hints
- Unit testing basics
- Virtual environment usage
- Git-based version control
- README documentation

Module 57: Core Python Interview Preparation

Core Python Interview Topics

- Python Features
- Interpreted Language Concepts
- Dynamic Typing
- Mutable vs Immutable Types
- Lists vs Tuples

- Sets vs Dictionaries
- Shallow vs Deep Copy
- `is` vs `==`
- `*args` vs `**kwargs`
- Scope and LEGB Rule
- Lambda Functions
- List Comprehensions
- Iterators
- Generators
- Decorators
- Context Managers
- Exception Handling
- Modules and Packages
- `__name__ == "__main__"`
- Classes and Objects
- Inheritance
- Polymorphism
- Encapsulation
- Abstract Base Classes
- Dunder Methods
- Class Methods vs Static Methods
- Dataclasses
- Closures
- Type Hints
- Virtual Environments
- Garbage Collection
- Reference Counting
- GIL Overview
- Threading vs Multiprocessing
- Async/Await Basics
- File Handling
- JSON and CSV
- Regular Expressions
- SQLite Basics
- Testing

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- Collection-Based Problems
- OOP Design Questions
- Function and Recursion Problems
- File Handling Problems
- Mock Interviews
- Timed Coding Tests

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs

- 20 String Programs
- 20 List, Tuple, Set, and Dictionary Exercises
- 15 Function and Recursion Programs
- 15 Comprehension and Functional Programming Exercises
- 20 OOP Programs
- 15 Exception Handling and File Programs
- 10 Module and Package Exercises
- 10 Iterator, Generator, and Decorator Exercises
- 10 Regular Expression Exercises
- 5 Database Exercises
- 5 Testing Exercises
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

- Section — Suggested Duration
- Python Fundamentals — 12–15 Hours
- Control Flow, Strings, and Collections — 20–25 Hours
- Functions, Modules, and Packages — 15–18 Hours
- Object-Oriented Programming — 18–22 Hours
- Exceptions, Files, CSV, and JSON — 12–15 Hours
- Iterators, Generators, Decorators, and Context Managers — 12–15 Hours
- Standard Library, Regex, and Type Hints — 12–15 Hours
- Testing, Debugging, Concurrency, and Async Basics — 12–15 Hours
- Database and Problem Solving — 10–12 Hours
- Projects and Interview Preparation — 20–25 Hours

Recommended Total Duration: approximately **145–175 hours**, depending on practical depth, assignments, and project work.

Prerequisites

No prior programming experience is required.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Learners who already know C, C++, Java, or C# may progress faster through the introductory modules.

Tools and Technologies

- Python 3
- CPython
- Visual Studio Code
- PyCharm
- IDLE
- Jupyter Notebook Overview
- Python REPL
- pip
- PyPI
- venv
- Git
- GitHub
- unittest
- pytest Overview
- SQLite

- Ruff / Flake8 Overview
- Black Overview
- Command Line / Terminal

Learning Outcomes

After completing the course, learners should be able to:

- Write structured and Pythonic applications.
- Understand Python data types, control flow, functions, and collections.
- Design modular applications using modules and packages.
- Apply object-oriented programming principles.
- Handle errors using robust exception handling.
- Work with files, CSV, JSON, and SQLite.
- Use comprehensions, iterators, generators, decorators, and context managers.
- Apply type hints and modern Python coding practices.
- Debug and test Python applications.
- Understand threading, multiprocessing, and async fundamentals.
- Use Git and GitHub professionally.
- Build portfolio-ready Python projects.
- Solve Core Python interview and coding problems.
- Progress confidently to specialized Python domains.

Recommended Progression After Core Python

After completing Core Python, learners can continue with:

- Advanced Python
- Data Structures and Algorithms in Python
- Python Automation and Scripting
- Web Development with Flask
- Web Development with Django
- FastAPI
- REST API Development
- SQL and Database Programming
- Data Analytics with Python
- NumPy
- Pandas
- Data Visualization
- Data Science
- Machine Learning
- Generative AI
- Agentic AI
- Test Automation with Python
- DevOps Automation
- Cloud Development
- Backend System Design with Python

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.